

LAB6: Randomized Greedy Knapsack with User Input

Write a Python program that performs a simplified greedy knapsack procedure using both **randomly generated items** and **one user-provided item**. You must include meaningful exception handling and custom sorting (no built-in sort functions).

Assignment Requirements

1. User Input for Number of Items

Your program must first ask the user: "How many items should be used?"

- The value must be an **integer** ≥ 2
- If the user enters:
 - A negative number
 - 0 or 1
 - A non-integer (e.g., "abc", "3.5")
→ You must trigger **ValueError** and handle it with try-except.

This number will determine total items.

Your program will randomly generate **(number_entered - 1)** items.

2. Random Item Generation

For the automatically produced items:

- Each item must be represented as a tuple:
(weight, value)
- Weight range: **1–20**
- Value range: **10–100**
- Generate **one fewer** than the number the user requested.

Example: user enters 5, your program generates **4 random items**.

3. One User-Provided Item (with validation)

Your program must then ask the user to enter:

- **weight**
- **value**

Both must be integers in the ranges:

- weight: **1–20**
- value: **10–100**

If the user enters:

- values outside the allowed ranges
- non-numeric input
→ Your program must raise and handle **ValueError** or **TypeError** in a try-except block.

This manually entered item is appended to the random items to complete the item list.

4. Random Knapsack Capacity

Generate a random knapsack capacity in the range:

- **20–60**
-

5. Custom Sorting (no sort or sorted)

You must sort the items by **value-to-weight ratio** in **descending order**, but:

- You may **NOT** use `sort()` or `sorted()`.
- You must implement your own sorting logic.

Sorting hint:

Repeatedly scan the list, compare each pair of neighboring items, and swap them if the next item has a higher value/weight ratio. Continue until the list is fully ordered.

6. Greedy Selection

After sorting:

- Pick items from highest ratio (value/weight). to lowest
- Add an item **only if** it fits in the remaining capacity.
- Stop when no more items can fit.

7. Output Requirements

Your program must print:

1. User input
2. Automatically generated items
3. User-provided item
4. Combined item list
5. Sorted list by ratio
6. Knapsack capacity
7. Selected items
8. Total weight and total value

Scenario	User Input	Random Items	Knapsack Capacity	Program Output / Notes
1. Normal execution	How many items? 5 Weight (last item)? 12 Value (last item)? 80	(5,40), (8,90), (3,20), (10,60)	40	Sorted by ratio: (8,90),(5,40),(3,20),(12,80),(10,60) Selected items: same as above Total weight: 38 Total value: 290
2. Knapsack small, only best ratios fit	How many items? 4 Weight? 12 Value? 60	(8,40), (5,30), (10,50)	15	Sorted by ratio: (5,30),(8,40),(12,60),(10,50) Selected items: (5,30), (8,40) Total weight: 13 Total value: 70
3. Only top item fits	How many items? 3 Weight? 15 Value? 90	(20,80),(10,50)	15	Sorted by ratio: (15,90),(10,50),(20,80) Selected items: (15,90) Total weight: 15 Total value: 90
4. Multiple items, maximize value/weight	How many items? 5 Weight? 8 Value? 100	(4,40), (6,50), (5,45), (7,60)	15	Sorted by ratio: (8,100),(7,60),(6,50),(5,45),(4,40) Selected items: (8,100),(7,60) Total weight: 15 Total value: 160
5. Edge case: total weight less than capacity	How many items? 4 Weight? 5 Value? 70	(2,30), (3,20), (4,25)	50	Sorted by ratio: (5,70),(2,30),(3,20),(4,25) Selected items: all items Total weight: 14 Total value: 145
6. Exception: negative number	How many items? -3	—	—	Invalid item count: You must enter at least 2.
7. Exception: user weight out of range	How many items? 4 Weight? 25 Value? 90	(5,30), (10,60), (7,50)	35	Invalid item provided: Weight out of range.
8. Exception: user value out of range	How many items? 4 Weight? 10 Value? 200	(5,30), (10,60), (7,50)	35	Invalid item provided: Value out of range.